

LangChain & Agentic AI – From Basics to Advanced demo-12 Notes

Requirement Analysis

Identify Input Variables

Design Prompt

Create Template

Objec(PromptTemplate/ChatPromptTemplate)

Get User Input

Invoke Template

Invoke LLM

AIMessage

Display Response

Application 1 — Restaurant Name Generator

Version 1 — Using PromptTemplate:

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import PromptTemplate
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
template = PromptTemplate.from_template(
```

```
    """
```

```
    Suggest 10 restaurant names.
```

```
    Cuisine: {cuisine}
```

```
    Location: {location}
```

```
    Style: {style}
```

```
    The names should be:
```

```
    - Attractive
```

- Easy to remember
- Suitable for the cuisine

""""

)

cuisine = input("Enter cuisine: ")

location = input("Enter location: ")

style = input("Enter restaurant style: ")

prompt = template.invoke(

{

"cuisine": cuisine,

"location": location,

"style": style

}

)

response = llm.invoke(prompt)

print("\nRestaurant Name Suggestions:")

```
print(response.content)
```

Version 2 — Using ChatPromptTemplate

Now let's implement the same application using chat messages.

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import ChatPromptTemplate
```

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
template = ChatPromptTemplate.from_messages(  
    [  
        (  
            "system",  
            "You are a creative branding expert "        )  
    ]  
)
```

```

        "who creates restaurant names."
    ),
    (
        "human",
        """

        Suggest 10 restaurant names.

        Cuisine: {cuisine}
        Location: {location}
        Style: {style}

        The names should be attractive
        and easy to remember.
        """
    )
]
)

```

```
cuisine = input("Enter cuisine: ")
```

```
location = input("Enter location: ")
```

```
style = input("Enter style: ")
```

```
prompt = template.invoke(  
    {  
        "cuisine": cuisine,  
        "location": location,  
        "style": style  
    }  
)
```

```
response = llm.invoke(prompt)
```

```
print(response.content)
```

```
-----
```

```
Application 2 — Travel Guide
```

```
-----
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import ChatPromptTemplate
```

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
template = ChatPromptTemplate.from_messages(  
    [  
        (  
            "system",  
            "You are a helpful travel planner. "  
            "Give practical and easy-to-follow plans."  
        ),  
        (  
            "human",  
            ""  
  
            Create a {days}-day travel plan for {place}.  
  
            Budget: {budget}  
            Main Interest: {interest}
```

Include:

1. Places to visit
2. Daily schedule
3. Food suggestions
4. Travel tips

""""

)

]

)

```
place = input("Enter destination: ")
```

```
days = input("Enter number of days: ")
```

```
budget = input("Enter budget: ")
```

```
interest = input("Enter main interest: ")
```

```
prompt = template.invoke(
```

```
{
```

```
    "place": place,
```

```
    "days": days,
```

```
        "budget": budget,  
        "interest": interest  
    }  
)
```

```
response = llm.invoke(prompt)
```

```
print("\nTravel Plan:")  
print(response.content)
```

```
-----
```

Application-3: Interview Helper Program

```
-----
```

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import ChatPromptTemplate
```

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")
```

)

```
template = ChatPromptTemplate.from_messages(  
    [  
        (  
            "system",  
            "You are an experienced technical interviewer."  
        ),  
        (  
            "human",  
            """"  
  
            Generate {count} {difficulty} interview questions  
            for a {level} candidate  
            on {technology}.  
  
            For every question:  
            - Give the question  
            - Give a simple answer  
            - Give one small example  
            """"
```

```
)  
]  
)
```

```
technology = input("Enter technology: ")  
level = input("Enter candidate level: ")  
count = input("Enter number of questions: ")  
difficulty = input("Enter difficulty: ")
```

```
prompt = template.invoke(  
    {  
        "technology": technology,  
        "level": level,  
        "count": count,  
        "difficulty": difficulty  
    }  
)
```

```
response = llm.invoke(prompt)
```

```
print("\nInterview Preparation:")  
print(response.content)
```

Application 4 — Blog Generator

```
import os
```

```
from langchain_openai import ChatOpenAI  
from langchain_core.prompts import ChatPromptTemplate
```

```
llm = ChatOpenAI(  
    model="gpt-4o-mini",  
    api_key=os.getenv("OPENAI_API_KEY")  
)
```

```
template = ChatPromptTemplate.from_messages(  
    [  
        (  
            "system",  
            "You are a professional content writer."        )  
    ]
```

),

(

"human",

""

Write a blog on {topic}.

Target Audience: {audience}

Tone: {tone}

Length: approximately {length}

Include:

- Attractive title
- Introduction
- Main sections
- Useful examples
- Conclusion

""

)

]

)

```
topic = input("Enter blog topic: ")
audience = input("Enter audience: ")
tone = input("Enter tone: ")
length = input("Enter approximate length: ")
```

```
prompt = template.invoke(
    {
        "topic": topic,
        "audience": audience,
        "tone": tone,
        "length": length
    }
)
```

```
response = llm.invoke(prompt)
```

```
print("\nGenerated Blog:")
print(response.content)
```

Application 5 — Smart Learning Assistant

Smart Learning Assistant — Complete Program

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.prompts import ChatPromptTemplate
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
template = ChatPromptTemplate.from_messages(
```

```
[
```

```
(
```

```
    "system",
```

```
    """
```

```
    You are an expert {subject} teacher.
```

Student Level: {level}

Language: {language}

Teaching Style: {style}

Always make the explanation
easy to understand.

""""

),

(

"human",

""""

Teach me {topic}.

Give:

1. A simple definition
2. Why it is required
3. Important points
4. {examples} simple examples
5. One common mistake
6. One-line conclusion

```
        """"  
    )  
]  
)
```

```
subject = input("Enter subject: ")  
topic = input("Enter topic: ")  
level = input("Enter student level: ")  
language = input("Enter language: ")  
style = input("Enter teaching style: ")  
examples = input("How many examples?: ")
```

```
prompt = template.invoke(  
    {  
        "subject": subject,  
        "topic": topic,  
        "level": level,  
        "language": language,  
        "style": style,  
        "examples": examples
```

```

    }
)

print("\n===== GENERATED MESSAGES =====")

for message in prompt.messages:
    print("\n", type(message).__name__)
    print(message.content)

response = llm.invoke(prompt)

print("\n===== AI RESPONSE =====")
print(response.content)

```

Mini Challenge 1 — Product Description Generator

Student requirement:

Inputs:

Product

Audience

Tone

Length

System:

You are a marketing copywriter.

Human:

Write a {length} product description

for {product}

targeting {audience}

using a {tone} tone.

Students should implement this independently.

Mini Challenge 2 — Job Preparation Assistant

Inputs:

Role

Experience

Days

Hours Per Day

Requirement:

Create a {days}-day preparation plan

for a {role} job

for a person with {experience} experience

who can study {hours} hours per day.

Use:

ChatPromptTemplate

Mini Challenge 3 — Email Assistant

Inputs:

Purpose

Recipient Type

Tone

Example:

Purpose = Request leave

Recipient Type = Manager

Tone = Professional

System:

You are a professional email-writing assistant.

Human:

Write an email for {purpose}

to my {recipient}
using a {tone} tone.

Mini Challenge 4 — Fitness Plan Concept

For classroom programming practice only:

Goal

Days

Experience Level

Template idea:

Create a general fitness routine
for someone with goal {goal},
training {days} days per week,
at {level} level.

{goal} → weight loss, muscle gain, general fitness, improve
stamina, improve flexibility

{days} → 2, 3, 4, 5, 6

{level} → beginner, intermediate, advanced

Build an AI Learning Plan Generator.

Inputs:

Technology

Current Level

Number of Days

Hours Per Day

Goal

Example:

Technology = Python

Current Level = Beginner

Days = 30

Hours Per Day = 2

Goal = Become job-ready

import os

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
```

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
template = ChatPromptTemplate.from_messages(
    [
        (
            "system",
            """
            You are an experienced technical trainer.
            Create practical learning plans
            that are easy to follow.
            """
        ),
        (
            "human",
```

""""

Create a learning plan for {technology}.

Current Level: {level}

Duration: {days} days

Available Time: {hours} hours per day

Goal: {goal}

Give:

- Day-wise or week-wise plan
- Important topics
- Practice suggestions
- Small projects
- Final revision strategy

""""

)

]

)

technology = input("Enter technology: ")

```
level = input("Enter current level: ")
days = input("Enter number of days: ")
hours = input("Enter hours per day: ")
goal = input("Enter goal: ")
```

```
prompt = template.invoke(
    {
        "technology": technology,
        "level": level,
        "days": days,
        "hours": hours,
        "goal": goal
    }
)
```

```
response = llm.invoke(prompt)
```

```
print("\nLearning Plan:")
print(response.content)
```

Common Mistake 1 — Starting Coding Without Prompt Design

Students may immediately write:

template = ...

Ayyooo!

First ask:

What is my requirement?

Then:

What changes?

Then:

What remains fixed?

Then design prompt.

Then code.

Common Mistake 2 — Making Everything Dynamic

Don't create variables just because you can.

Bad:

{word1}

{word2}

{instruction1}

{instruction2}

{symbol}

unless those really need to change.

Keep templates simple.

Common Mistake 3 — Variable Name Mismatch

Template:

Create a plan for {place}

Input:

```
{  
  "city": "Goa"  
}
```

Wrong.

Correct:

```
{  
  "place": "Goa"  
}
```

Common Mistake 4 — Missing Input

Template expects:

place

days

budget

But dictionary provides:

place

days

budget is missing.

You can inspect:

```
print(template.input_variables)
```

Common Mistake 5 — Confusing Template Invocation and Model Invocation

Again:

```
template.invoke(...)
```

means:

Create formatted prompt/messages

Whereas:

`llm.invoke(...)`

means:

Call the model

Very important.

Common Mistake 6 — Hardcoding API Key

Avoid:

```
api_key="sk-xxxxxxx"
```

Prefer:

```
api_key=os.getenv("OPENAI_API_KEY")
```

The prompt should not contain API secrets.

Common Mistake 7 — Assuming PromptTemplate Is Always Required

Remember:

```
topic = "Python"
```

```
prompt = f"Explain {topic}"
```

is perfectly valid Python.

For a tiny application, an f-string may be enough.

We use LangChain prompt abstractions when they help us build reusable and structured LLM workflows.